

Paso 03 · La capa Staging con dbt

Qué construimos, cómo funciona y por qué — el primer contacto con dbt

Mathias Berg · Stuttgart · 2026-07-08 · Repo: github.com/mbergr/jazz-standards-analytics

01 Qué es la capa staging (y qué es dbt)

dbt convierte consultas SQL (SELECT) en un pipeline de modelos versionado, testeado y documentado. Cada modelo es un archivo `.sql` con un SELECT; dbt se encarga de crear la vista o tabla en la base, en el orden correcto según sus dependencias.

La **capa staging** es la primera transformación. Sus modelos son **1:1 con cada fuente** y solo hacen limpieza: recortar espacios, renombrar columnas a un estándar, castear tipos y normalizar valores. **Nada de lógica de negocio** (joins, deduplicación, métricas) todavía — eso viene en las capas intermediate y marts. Pensala como el “espejo ordenado” del crudo sobre el que se apoya todo lo demás.

La regla de oro del staging

Un modelo staging por cada tabla de origen, mismo grano que la fuente, solo renombrado / casteo / limpieza. Si estás por escribir un JOIN, ya no es staging.

02 La estructura que creamos

Todo vive en `dbt_project/`:

```
dbt_project/
|- dbt_project.yml      # config: nombre del proyecto, staging = vistas
|- profiles.yml         # conexion a ../data/jazz.duckdb (DuckDB local)
\-- models/
    \-- staging/
        |- _sources.yml  # declara las fuentes raw.standards / raw.recordings
        |- stg_standards.sql # modelo: limpieza del canon
        |- stg_recordings.sql # modelo: limpieza de grabaciones
        \-- _staging_models.yml # documentacion + tests de los modelos
```

Archivo	Rol
<code>dbt_project.yml</code>	Config del proyecto. Define que los modelos de staging se materializan como vistas.
<code>profiles.yml</code>	La conexion. Apunta al archivo DuckDB. Sin secretos → se puede commitear.
<code>_sources.yml</code>	Le dice a dbt que existen <code>raw.standards</code> y <code>raw.recordings</code> (las tablas de la ingesta).

03 Las fuentes (sources)

Antes de transformar, dbt necesita saber de dónde salen los datos. En `_sources.yml` declaramos las dos tablas crudas que dejó la ingesta. A partir de ahí, los modelos las referencian con la función `{{ source(...) }}` en vez de escribir el nombre de la tabla a mano — así dbt entiende las dependencias y puede dibujar el linaje.

```
sources:
- name: raw
  schema: raw
  tables:
    - name: standards      # 100 standards del scraping
    - name: recordings      # 703 grabaciones candidatas de Spotify
```

04 Los dos modelos de staging

stg_standards — limpia el canon. Renombra a nombres estándar, recorta espacios y convierte strings vacíos en NULL:

```
with source as (
  select * from {{ source('raw', 'standards') }}
),
cleaned as (
  select
    slug                as standard_id,      -- clave natural
    rank                as popularity_rank,
    trim(title)         as title,
    nullif(trim(composer), '') as composer,
    year               as composed_year,
    ...
  from source
)
select * from cleaned
```

stg_recordings — limpia las grabaciones. El detalle más interesante es normalizar la fecha: `album_release_date` viene en granularidad mezclada (a veces "1993", a veces "2011-09-20"). Los primeros 4 caracteres siempre son el año:

```
standard_slug      as standard_id,  -- FK a stg_standards
spotify_track_id  as track_id,
trim(track_name)   as track_name,
-- de "1993" o "2011-09-20" -> 1993 (entero robusto)
try_cast(left(album_release_date, 4) as integer) as release_year,
round(duration_ms / 1000.0)              as duration_seconds,
```

Resultado verificado: `release_year` parsea las dos granularidades correctamente, con **0 nulls en 703 filas**.

05 Los tests

dbt corre **tests** (aserciones sobre los datos) en cada build. Si un test falla, el pipeline se detiene. Definimos 7, todos nativos de dbt (sin paquetes externos):

Modelo	Columna	Test	Qué garantiza
stg_standards	standard_id	unique + not_null	Cada standard tiene una clave única y presente
stg_standards	title	not_null	Ningún standard sin título
stg_standards	popularity_rank	not_null	Todos tienen ranking
stg_recordings	standard_id	not_null	Ninguna grabación huérfana
stg_recordings	standard_id	relationships	Cada grabación apunta a un standard que existe

stg_recordings	track_id	not_null	Toda grabación tiene id de Spotify
----------------	----------	----------	------------------------------------

El test de **relationships** es el más valioso: es una comprobación de integridad referencial (como una foreign key), que confirma que no quedaron grabaciones apuntando a un standard inexistente.

06 El resultado: dbt build

Un solo comando construye los modelos y corre los tests en orden de dependencias:

```
cd dbt_project
dbt build --profiles-dir .

OK created sql view model main.stg_standards ..... [OK]
OK created sql view model main.stg_recordings ..... [OK]
PASS not_null_stg_standards_standard_id ..... [PASS]
PASS unique_stg_standards_standard_id ..... [PASS]
PASS relationships_stg_recordings_standard_id ..... [PASS]
... (9 nodos)
Done. PASS=9 WARN=0 ERROR=0
```

07 Decisiones conscientes

Staging como vistas, no tablas

Los modelos se materializan como VIEW: se recalculan al vuelo, siempre reflejan el crudo y no duplican datos. Para volúmenes chicos es lo ideal; las tablas físicas se reservan para las marts.

Sin dbt_utils todavía

Mantuvimos el primer contacto sin paquetes externos, usando solo tests nativos. dbt_utils (surrogate keys, uniqueness combinada) entra en el paso 04, donde realmente hace falta.

No separamos compositores múltiples aca

El crudo perdió el delimitador entre coautores (los
 se volvieron espacios) y un nombre propio también lleva espacio, así que separarlos de forma confiable no es posible desde staging. Se resolverá con un ajuste del scraper + una tabla puente más adelante.

08 Glosario dbt

Término	Qué significa
Model (modelo)	Un archivo .sql con un SELECT. dbt lo materializa como vista o tabla en la base.
Source (fuente)	Una tabla cruda declarada en YAML, referenciada con {{ source(...) }}.
Materialization	Cómo se persiste un modelo: view (vista) o table (tabla física), entre otras.
Staging	Primera capa: limpieza 1:1 con cada fuente, sin logica de negocio.
Test	Aserción sobre los datos (not_null, unique, relationships) que corre en cada build.
dbt build	Construye los modelos y corre sus tests en orden de dependencias.
Profile	La configuración de conexión a la base (en profiles.yml).
Lineage	El grafo de dependencias entre fuentes y modelos, que dbt deriva automáticamente.

